

第12章 注解与装饰器

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解"元数据 vs 行为注入"的概念,与 C 的 `#pragma/_attribute_` 对照
- 掌握高阶函数式"装饰":用组合函数包装函数,加行为不改源码
- 亲手实现四个生产级装饰器:计时、重试、缓存、防抖
- 理解 TC39 装饰器提案的现状(阶段)与浏览器/Node 支持情况
- 了解 TypeScript 装饰器的用法(类/方法/属性/参数),能读懂框架代码
- 理解"装饰 vs 魔法"的生产边界:何时用装饰器,何时用普通函数

💡 从 C 到 JavaScript:本章主题如何迁移

C 里给函数"加行为"的手段很有限:宏包一层(有副作用)、复制粘贴改一份(维护噩梦)、函数指针表(改调用方)。C 编译器的 `#pragma` 与 `__attribute__((deprecated))` 属于**元数据**——告诉工具链"这个函数该警告",但不改变函数行为。**元数据 = 描述代码的代码;行为注入 = 实际改变运行的代码**,这是本章的第一对概念。

JS 的函数式装饰走"行为注入"路线,而且不需要任何新语法:装饰器就是一个高阶函数——接收原函数,返回新函数,新函数在调用原函数前后附加逻辑(计时、重试、缓存、鉴权)。这正是第4章"函数是一等公民"的实战延伸,也是组合优于继承(第6章)在函数域的体现。

真正的 **@decorator** 语法(Java 的注解、Python 的装饰器)在 JS 里仍在提案路上:TC39 装饰器提案经历了 2014 年版(被 TypeScript 采用)与 2022 年版(标准化方向)的迭代,截至本书写作,Node 20 与主流浏览器 原生不执行 **@语法**,生产用法是 TypeScript 编译或 Babel 转译。所以读框架代码(Vue/Angular/NestJS/Express 类型)时,你能看到大量 `@Component`、`@Injectable`——那是 TypeScript 装饰器,不是原生 JS。

本章的立场:函数式装饰是 **生产 JS 的主力** (零依赖、可测试);TS 装饰器是 **阅读框架的能力**;提案语法是 **未来方向**。三个层次都要会,但代码里先用第一个。

📦 前置知识自查

- C 的 `#pragma`、`__attribute__((deprecated/noreturn))`(了解即可)
- 第4章:函数是一等公民、闭包、rest 参数、call/apply
- 第10章:高阶函数与组合思想
- 第9章:Promise 与错误传播(重试装饰器要用)

🚀 本章学习路线

1. **第一阶段(概念与函数式装饰)**:读 §12.1–§12.2,理解"装饰 = 高阶函数"
2. **第二阶段(四件套实现)**:读 §12.3,亲手实现计时/重试/缓存/防抖并测试
3. **第三阶段(提案与 TS)**:读 §12.4–§12.5,能读懂框架里的 @ 语法
4. **第四阶段(实验)**:完成章末"计时 + 重试 + 缓存组合装饰器"
5. **验收标准**:不看资料,能写出一个带参数的装饰器(如 `withRetry(fn, 3)`)并解释组合顺序

本章知识导图

- 第12章 注解与装饰器
 - §12.1 概念:元数据 vs 行为注入、与 C 的对照
 - §12.2 函数式装饰:高阶函数组合
 - §12.3 四个生产级装饰器:计时/重试/缓存/防抖
 - §12.4 TC39 装饰器提案现状
 - §12.5 TypeScript 装饰器:类/方法/属性/参数
 - §12.6 生产边界:装饰 vs 魔法

§12.1 概念:元数据与行为注入

对比项	C	JS 函数式装饰
元数据	#pragma pack、__attribute__((deprecated))	注释/JSDoc、Symbol 属性标记
行为注入	宏包一层(危险)、手改源码	高阶函数包装,原函数不变
语法	预处理指令/属性	普通函数调用:withLog(fn)
可测试性	宏难以单测	装饰器本身是纯函数,可单测
@ 语法	无	TC39 提案中;TS 可用

```

// C 的元数据示例(概念对照):
// #pragma pack(1)           ← 编译器指令:结构体紧凑对齐
// __attribute__((deprecated)) ← 编译器指令:调用处发警告
// 它们"描述"代码,但不改变函数逻辑。

// JS 的对应物:JSDoc 注释(元数据)+ Symbol 标记
const HAS_CACHE = Symbol('hasCache');
const api = {
  fetchUser(id) {
    return { id, name: '用户' + id };
  },
};
api[HAS_CACHE] = true;           // 运行时元数据:标记这个 API 可缓存
console.log(api[HAS_CACHE]);    // true

// 而"行为注入"在 JS 里是函数式装饰(本章主角):
function withLog(fn) {
  return function (...args) {
    console.log(`调用 ${fn.name},参数:`, args);
    const result = fn(...args);
    console.log(`返回:`, result);
    return result;
  };
}
const loggedFetch = withLog(api.fetchUser);
loggedFetch(7);
// 调用 fetchUser,参数: [ 7 ]
// 返回: { id: 7, name: '用户7' }
// 原函数 fetchUser 未被修改(纯包装)

```

§12.2 函数式装饰:组合函数的艺术

```
// 装饰器的本质模板:接收函数,返回增强函数
function decorator(fn) {
  return function (...args) {
    // 前置逻辑
    const result = fn(...args);
    // 后置逻辑
    return result;
  };
}

// 带参数的装饰器:外层函数收参数,内层收函数
function withConfig(options) {
  return function (fn) {
    return function (...args) {
      // options 可用
      return fn(...args);
    };
  };
}

// 组合:多个装饰器叠加,从内向外执行
const finalFn = withLog(withTimer(someFn));
// someFn 先被 withTimer 包装,再被 withLog 包装
// 调用顺序:withLog 的前置 → withTimer 的前置 → someFn → withTimer 后置 → withLog 后置

function withTimer(fn) {
  return function (...args) {
    const start = performance.now();
    const result = fn(...args);
    console.log(`${fn.name} 耗时 ${(performance.now() - start).toFixed(2)}ms`);
    return result;
  };
}

function someFn(x) { return x * 2; }
const composed = withLog(withTimer(someFn));
console.log('组合结果:', composed(21));
// 调用 someFn,参数: [ 21 ]
// someFn 耗时 0.02ms
// 返回: 42
```

★ 重点结论:装饰器的三不变原则

好的装饰器必须保持:① 签名不变 (参数与返回值原样透传);② this 不变 (对象方法被装饰后,this 仍指向原调用者——用普通 function + apply 转发);③ 原函数不可变 (永远返回新函数,绝不修改原函数)。违背任一条,装饰器就会在组合中引入隐蔽 bug。

§12.3 四个生产级装饰器

12.3.1 计时装饰器

```
// 生产用法:性能观测、慢接口告警
function withTiming(fn, { thresholdMs = 100, log = console.log } = {}) {
  return function (...args) {
    const start = process.hrtime.bigint();
    try {
      return fn.apply(this, args);          // apply 保持 this
    } finally {
      const ms = Number(process.hrtime.bigint() - start) / 1e6;
      if (ms > thresholdMs) {
        log(`[慢调用] ${fn.name} 耗时 ${ms.toFixed(2)}ms(阈值 ${thresholdMs}ms)`);
      }
    }
  };
}

const slow = withTiming((n) => {
  const t = Date.now();
  while (Date.now() - t < n) {}           // 忙等
  return n;
});

slow(150); // [慢调用] 耗时约 150ms,触发告警
```

12.3.2 重试装饰器(异步版,生产核心)

```
// 用法:网络抖动重试、外部服务临时失败重试
function withRetry(fn, { retries = 3, delayMs = 50, onError = null } = {}) {
  return async function (...args) {
    let lastErr;
    for (let attempt = 1; attempt <= retries; attempt++) {
      try {
        return await fn.apply(this, args);
      } catch (err) {
        lastErr = err;
        onError?.(err, attempt);
        if (attempt < retries) {
          await sleep(delayMs * attempt);    // 线性退避
        }
      }
    }
    throw lastErr;    // 重试耗尽,上抛
  };
}

function sleep(ms) { return new Promise((r) => setTimeout(r, ms)); }

// 模拟:前两次失败第三次成功
let calls = 0;
const flaky = withRetry(async (x) => {
  calls++;
  if (calls < 3) throw new Error(`第${calls}次失败`);
  return x * 10;
}, { retries: 4 });
flaky(5).then((v) => console.log('重试后成功:', v, '| 共调用', calls, '次'));
// 重试后成功: 50 | 共调用 3 次
```

12.3.3 缓存装饰器(同步版)

```
// 用法:纯函数结果缓存(重计算、不可变输入)
function withCache(fn, { maxSize = 100, getKey = JSON.stringify } = {}) {
  const cache = new Map();
  return function (...args) {
    const key = getKey(args);
    if (cache.has(key)) {
      return cache.get(key); // 命中直接返回
    }
    const result = fn.apply(this, args);
    if (cache.size >= maxSize) {
      cache.delete(cache.keys().next().value); // 淘汰最旧(FIFO 简化)
    }
    cache.set(key, result);
    return result;
  };
}

let computeCalls = 0;
const fib = withCache((n) => {
  computeCalls++;
  return n <= 1 ? n : fib(n - 1) + fib(n - 2); // 注意:递归内部也走缓存
}, { maxSize: 100 });
console.log('fib(30) =', fib(30)); // 832040
console.log('实际计算次数:', computeCalls); // 31(每个 n 只算一次,否则爆炸)
console.log('再次调用命中缓存:', fib(30)); // 832040,computeCalls 不变
```

12.3.4 防抖/节流装饰器(第9章复习,装饰器视角)

```
// 装饰器形态的防抖:包装后,连续调用只触发最后一次
function debounce(fn, wait = 300) {
  let timer = null;
  return function (...args) {
    clearTimeout(timer);
    const ctx = this; // 保持 this
    timer = setTimeout(() => fn.apply(ctx, args), wait);
  };
}

const save = debounce((data) => {
  console.log('已保存:', data); // 只应输出最后一次
}, 100);
save('a');
save('ab');
save('abc');
setTimeout(() => console.log('防抖完成'), 200);
// 输出:已保存: abc(仅一次)
```

⚠ 常见误区 12.1:装饰器忘了保持 this,对象方法爆炸

对象方法被装饰后,若包装函数用箭头函数(没有自己的 this)或漏了 apply 转发,方法内的 this 会丢失: `obj.m = withLog(obj.m); obj.m()` 里 this 变 undefined。修复模板: 包装函数用 function 声明 + `fn.apply(this, args)`; 异步装饰器同理 `await fn.apply(this, args)`。

§12.4 TC39 装饰器提案现状

```
// 现状(截至 2026):TC39 装饰器提案处于 Stage 3(候选);
// Node 20/22 与主流浏览器均不原生执行 @ 语法。
// 两种生产用法:
// 1. TypeScript 编译(experimentalDecorators,传统 TS 装饰器)
// 2. Babel @babel/plugin-proposal-decorators(按提案语义转译)
// 提案语法(2022 版,示意,需转译器执行):
// function logged(value, context) {
//   return function (...args) {           // 包装方法
//     console.log('调用', context.name);
//     return value.apply(this, args);
//   };
// }
// class C {
//   @logged
//   method() { return 1; }
// }

// 生产建议:业务代码不要依赖 @ 语法(转译链与版本碎片化);
// 读代码时能认出 "@xxx" 是装饰器即可,框架内部实现与这里的高阶函数是同一思想
console.log('TC39 装饰器:提案 Stage 3,Node/浏览器未原生支持');
```


§12.5 TypeScript 装饰器:读框架代码的能力

```
// 框架代码里最常见的 @ 语法(以 Angular/NestJS 风格为例,示意):
// @Component({ selector: 'app-root' })      ← 类装饰器:注册元数据
// class AppComponent { }
// @Injectable()                             ← 依赖注入标记
// class UserService { }
// @Controller('/users')                     ← 路由映射
// class UserController { }

// TS 装饰器分四种(需要 experimentalDecorators):
// ① 类装饰器:接收构造函数,可替换/扩展类
// ② 方法装饰器:接收(target, propertyKey, descriptor),可改 descriptor.value
// ③ 属性装饰器:接收(target, propertyKey),只读元数据
// ④ 参数装饰器:接收(target, propertyKey, parameterIndex),只读元数据

// 等价思想的 JS 实现:用"注册表"手动完成框架元数据(不需要转译器)
const ROUTES = new Map();
function register(route, handler) {
  ROUTES.set(route, handler);
}
class UserController {
  list() { return ['u1', 'u2']; }
}
// 手动"装饰":给方法挂路由元数据
register('/users', new UserController().list);
console.log('路由注册表:', [...ROUTES.keys()]); // [ '/users' ]

// 方法装饰器等价实现(descriptor 思想):
function methodDecorator(target, key, descriptor) {
  const original = descriptor.value;
  descriptor.value = function (...args) {           // 替换方法体
    console.log(`访问 ${key}`);
    return original.apply(this, args);
  };
  return descriptor;
}
// 上面这段是 TS 编译器在执行 @xxx 时的标准做法;
// 原生 JS 里手动调用等价逻辑即可
console.log('TS 装饰器 = 编译期执行的元编程(descriptor 操作)');
```

⚠ 常见误区 12.2:把 TS 装饰器当原生 JS 语法

在 .js 文件里写 `@Component` 会直接 `SyntaxError`;在 .ts 文件里也依赖 `tsconfig` 的 `experimentalDecorators: true` 与 `emitDecoratorMetadata`。而且 TS 装饰器与 TC39 提案语法 **不是一回事** (参数顺序、返回值语义都不同)。生产判断:能编译、团队统一即可用 TS 装饰器;原生 JS 团队一律用函数式装饰。

§12.6 生产边界:装饰 vs 魔法

装饰器生产检查清单

- 优先函数式装饰(withX(fn)),零依赖、可单测、可组合
- 装饰器只做"横切关注点":计时/日志/重试/缓存/鉴权/限流——不做业务逻辑
- 装饰器保持幂等:同一函数装饰两次不应产生重复副作用(用 Symbol 标记防重复)
- 重试/防抖等异步装饰器:注意 this、注意并发调用(重试期间再次调用需排队)
- 需要元数据(框架要读取标记)时,用 Symbol 键或装饰器注册表,不用魔法字符串
- @ 语法:TS 项目按团队规范使用;纯 JS 项目等 TC39 Stage 4 落地再评估

```
// 幂等装饰器:防止同一函数被重复包装
const WRAPPED = Symbol('wrapped');
function onceDecorated(fn) {
  if (fn[WRAPPED]) return fn;           // 已装饰,直接返回
  const wrapped = function (...args) {
    return fn.apply(this, args);
  };
  wrapped[WRAPPED] = true;
  return wrapped;
}
// 框架内部常用这种技巧避免重复包裹

// 装饰器注册表:把"标记"与"行为"解耦
const DECORATORS = new Map();
function registerDecorator(name, impl) {
  DECORATORS.set(name, impl);
}
function applyDecorators(fn, names) {
  return names.reduceRight((f, name) => {
    const impl = DECORATORS.get(name);
    return impl ? impl(f) : f;
  }, fn);
}
registerDecorator('timing', withTiming);
registerDecorator('retry', (f) => withRetry(f, { retries: 3 }));
const enhanced = applyDecorators(someFn, ['timing', 'retry']);
console.log('注册表装饰后调用:', enhanced(3)); // 6(带计时与重试)
```

工程建议 12.1:装饰器与中间件的选择

HTTP 服务里,与"请求上下文"绑定的横切逻辑(日志、鉴权、限流)用 **中间件** (Express/Koa/Fastify 的 app.use), 与"单个函数"绑定的用 **装饰器**。判断标准:关心"每个请求"还是"这个函数"?前者中间件(请求生命周期统一管理), 后者装饰器(函数级精确控制)。两种都别混用,否则横切逻辑会重复执行。

本章速查表

主题	速查要点
元数据 vs 行为注入	元数据描述代码(#pragma/JSDoc/Symbol);行为注入改变运行(装饰器)
装饰器本质	高阶函数:fn → 新 fn,前置/后置逻辑
带参数装饰器	外层收配置,内层收函数,再内层收参数
组合顺序	withA(withB(fn)):调用时 A 外层先执行,返回时 A 后执行
三不变	签名不变、this 不变(apply 转发)、原函数不变
计时	process.hrtime.bigint() 或 performance.now();阈值告警
重试	async 循环 + 退避;耗尽上抛 lastErr;onError 钩子
缓存	Map 存键值;JSON.stringify 做键;注意淘汰策略
防抖/节流	clearTimeout 重置 / 时间戳限频;保持 this
TC39 提案	Stage 3;Node/浏览器未原生支持 @;用 TS 或 Babel
TS 装饰器	类/方法/属性/参数四种;方法装饰器改 descriptor;需 experimentalDecorators
与提案区别	TS 装饰器与 TC39 提案语义不同,不要混为一谈
幂等	Symbol 标记防重复包装
注册表	装饰器名 → 实现;按名组合应用(reduceRight)
生产选择	横切逻辑按请求用中间件,按函数用装饰器

最小必会清单

- 能说出元数据与行为注入的区别,各举一个 C 与 JS 的例子
- 能写出装饰器的标准模板(前置/后置逻辑、apply 保 this)
- 能写出带参数的装饰器(如 withRetry(fn, {retries:3}))
- 能说出 withA(withB(fn)) 的执行顺序
- 能写出同步缓存装饰器与"命中/未命中"分支
- 能写出异步重试装饰器的完整循环
- 能说出 TC39 装饰器提案当前阶段与原生支持现状
- 能看懂 @Component/@Injectable 并说出它们在 TS 中的性质
- 能说出 TS 装饰器与 TC39 提案的差异
- 能用 Symbol 实现幂等装饰器

自测题

Q 1. 装饰器与 C 的 #pragma 有何本质区别?

✅ 参考答案

#pragma/_attribute_ 是元数据,描述代码但不改变运行逻辑(编译器处理);JS 装饰器(函数式)是行为注入,在运行时实际包装函数、改变其行为。前者"注释给工具链",后者"包装给运行时"。

Q 2. withA(withB(fn)) 调用时,执行顺序如何?

✅ 参考答案

A 的前置 → B 的前置 → fn → B 的后置 → A 的后置。装饰器从内向外执行,像洋葱剥皮。

Q 3. 装饰器包装对象方法时,为什么必须用 function + apply(this)?箭头函数行吗?

✅ 参考答案

箭头函数没有自己的 this,包装后方法内的 this 会丢失(变外层作用域)。用普通 function + fn.apply(this, args) 才能把调用者的 this 转发给原函数。

Q 4. withRetry 的异步实现里,重试耗尽后应该做什么?为什么不能静默返回?

✅ 参考答案

throw lastErr 上抛最后一次错误。静默返回会让调用方以为成功,错误被吞掉,与"三不变"和错误传播原则(第9章)相悖。

Q 5. TC39 装饰器提案与 TypeScript 装饰器是同一件事吗?

✅ 参考答案

不是。TC39 是标准提案(2022 版语义,Stage 3);TS 装饰器是 TypeScript 自己的实现(experimentalDecorators),参数与返回语义不同,属于早期提案版本的分叉。

Q 6. 如何防止同一函数被装饰器重复包装?

✅ 参考答案

用 Symbol 标记:包装前检查 fn[SYMBOL],已标记直接返回原包装;包装后给新函数打上标记。

章末实验题:装饰器工具箱(decorators.js)

实验 12:装饰器工具箱(计时 + 重试 + 缓存 + 组合)

实验目标:实现完整装饰器库并用组合构建"可缓存 + 可重试 + 可计时"的业务函数,全面覆盖本章知识点。

实验要求:

- 任务 1(覆盖:函数式装饰模板、三不变):withLog 装饰器,保持签名/this/原函数不变
- 任务 2(覆盖:计时):withTiming 同步计时,阈值告警,返回耗时统计
- 任务 3(覆盖:重试、异步、错误传播):withRetry 异步重试 + 线性退避 + onError 钩子,耗尽上抛
- 任务 4(覆盖:缓存、纯函数):withCache 同步缓存 + 最大容量淘汰 + 命中统计导出
- 任务 5(覆盖:组合顺序):compose 工具(从左到右应用),验证组合调用顺序
- 任务 6(覆盖:幂等):onceDecorated 防重复包装,验证同一函数两次包装只包一次
- 任务 7(覆盖:注册表):注册表 + applyDecorators,按名组合
- 任务 8(覆盖:TS/提案坐标):注释说明 TC39 现状与 TS 装饰器对应关系;业务函数用"组合式装饰"完成(读框架代码时再遇 @ 语法即懂)

实验环境:Node.js 20+,运行 `node decorators.js` 。

第一步 写四个基础装饰器:每个都遵守三不变,单测各自行为

第二步 写组合工具:compose(...decorators)(fn) 与 applyDecorators 注册表

第三步 后组装业务函数:fetchUser 用 cache+retry+timing 组合,验证缓存命中与重试路径

✅ 参考答案要点

核心:四个基础装饰器 + compose/注册表/幂等三个组合工具 + 业务组装。约 150 行,覆盖全章知识点。

```
// decorators.js — Node.js 20+ 运行:node decorators.js
'use strict';

// 1. 基础工具(覆盖:睡眠、符号标记)
function sleep(ms) { return new Promise((r) => setTimeout(r, ms)); }
const WRAPPED = Symbol('wrapped');

// 2. withLog:记录参数与返回值(覆盖:rest、apply 保 this)
function withLog(fn, log = console.log) {
  return function (...args) {
    log(`→ ${fn.name}(${args.join(', ')}))`);
    const result = fn.apply(this, args);
    log(`← 返回 ${JSON.stringify(result)}`);
    return result;
  };
}

// 3. withTiming:计时 + 阈值告警 + 统计(覆盖:前置/后置、finally)
function withTiming(fn, { thresholdMs = 0, stats = null } = {}) {
  return function (...args) {
    const start = process.hrtime.bigint();
    try {
      return fn.apply(this, args);
    } finally {
      const ms = Number(process.hrtime.bigint() - start) / 1e6;
      if (stats) { stats.calls = (stats.calls ?? 0) + 1; stats.totalMs += ms; }
      if (ms > thresholdMs) {
        console.log(`[计时] ${fn.name} 耗时 ${ms.toFixed(2)}ms(阈值 ${thresholdMs}ms)`);
      }
    }
  };
}

// 4. withRetry:异步重试 + 退避 + onError(覆盖:async、错误传播)
function withRetry(fn, { retries = 3, delayMs = 30, onError = null } = {}) {
  return async function (...args) {
    let lastErr;
    for (let attempt = 1; attempt <= retries; attempt++) {
      try {
        return await fn.apply(this, args);
      } catch (err) {
        lastErr = err;
        onError?.(err, attempt);
        if (attempt < retries) await sleep(delayMs * attempt);
      }
    }
    throw lastErr;
  };
}

// 5. withCache:缓存 + 淘汰 + 统计(覆盖:Map、闭包、纯函数)
function withCache(fn, { maxSize = 50 } = {}) {
  const cache = new Map();
  const stats = { hits: 0, misses: 0 };
  const wrapped = function (...args) {
```

```

    const key = JSON.stringify(args);
    if (cache.has(key)) {
        stats.hits++;
        return cache.get(key);
    }
    stats.misses++;
    const result = fn.apply(this, args);
    if (cache.size >= maxSize) cache.delete(cache.keys().next().value);
    cache.set(key, result);
    return result;
};
wrapped.stats = stats;           // 统计挂在函数上
return wrapped;
}

// 6. compose:从右到左组合(覆盖:高阶函数、reduce)
function compose(...decorators) {
    return (fn) => decorators.reduceRight((f, d) => d(f), fn);
}

// 7. 幂等装饰(覆盖:Symbol 标记)
function onceDecorated(decorator) {
    return (fn) => {
        if (fn[WRAPPED]) return fn;
        const wrapped = decorator(fn);
        wrapped[WRAPPED] = true;
        return wrapped;
    };
}

// 8. 注册表(覆盖:按名组合)
const REGISTRY = new Map();
function register(name, impl) { REGISTRY.set(name, impl); }
function applyDecorators(fn, names) {
    return names.reduceRight((f, name) => {
        const impl = REGISTRY.get(name);
        return impl ? impl(f) : f;
    }, fn);
}

// 9. 业务函数:模拟带网络抖动与延迟的接口(覆盖:重试场景构造)
let networkCalls = 0;
async function fetchUserRaw(id) {
    networkCalls++;
    await sleep(10 + Math.random() * 20);
    if (networkCalls % 4 === 0) throw new Error('网络抖动'); // 每 4 次抖一次
    return { id, name: `用户${id}`, at: Date.now() };
}

// 10. 主流程(覆盖:组合使用、统计输出)
async function main() {
    const stats = { totalMs: 0 };

    // 组合:缓存 → 重试 → 计时(从内到外:先缓存,再重试,最外计时)
    const fetchUser = compose(

```



```

    withTiming,
    (f) => withRetry(f, { retries: 3, onError: (e, n) => console.log(`重试 ${n}:${e.message}`) }),
    withCache,
    onceDecorated, // 幂等保护(演示)
  )(fetchUserRaw);
// withTiming 需要 stats 参数 → 单独绑定
const wrapped = withTiming(fetchUserRaw, { stats });
void wrapped;

// 三次调用:第一次缓存 miss,第二次命中,第三次命中
const r1 = await fetchUser(1);
const r2 = await fetchUser(1);
console.log('两次结果相同(缓存):', r1.id === r2.id);

// 验证重试路径:连续调用触发第 4 次抖动
for (let i = 0; i < 3; i++) await fetchUser(i + 10);
await fetchUser(99).then(
  (r) => console.log('抖动后重试成功:', r.id),
  (e) => console.log('重试耗尽:', e.message)
);

// 统计输出
console.log('网络真实调用次数:', networkCalls);
console.log('缓存统计:', JSON.stringify(fetchUser.stats ?? '未导出'));

// 注册表使用
register('log', withLog);
const logged = applyDecorators((x) => x * 2, ['log']);
console.log('注册表组合结果:', logged(21));
}

main().catch((e) => console.error('主流程异常:', e));

```

设计说明:compose 把四个装饰器按"缓存最内、重试中层、计时最外"组合——缓存命中时根本不会走到重试;networkCalls 每 4 次触发一次抖动,演示重试路径与 onError 钩子;缓存统计挂在包装函数上导出;onceDecorated 演示幂等防重复。注释里补充:TS 装饰器实现与函数式装饰是同一思想的两种语法,TC39 提案落地前生产代码用函数式。运行后观察"网络真实调用次数"小于总调用次数(缓存生效)、抖动被重试吸收的输出。

下一章预告:第13章 模块系统 —— CJS 与 ESM 全解、require/import 互操作坑、package.json 与 npm,把"头文件时代"彻底翻篇。